AWK

May 2, 2026

Canine-Table

POSIX Nexus serves as a comprehensive cross-language reference hub that explores the implementation and behavior of POSIX-compliant functionality across a diverse set of programming environments. Built atop the foundational IEEE Portable Operating System Interface (POSIX) standards, this project emphasizes compatibility, portability, and interoperability between operating systems.

Abstract

Contents

I Math Module

I Signed Number Detection

__nx_is_signed(N)

```awk
function __nx_is_signed(N)
{
    return N ~ /^([-]|[+])/
}
```

Signed Number Detection — The Prefix Glyph

- **Purpose** ⇒ Detects if N begins with a sign prefix
- **Regex** ⇒ `^([-]|[+])` matches optional + or - at start
- **Use Case** ⇒ Pre-validation for signed numeric parsing or formatting

I Integer Format Validator

__nx_is_integral(N, B)

```awk
function __nx_is_integral(N, B)
{
    return __nx_if(B, N ~ /^([-]|[+])?[0-9]+$/, N ~ /^[0-9]+$/)
}
```

Integer Format Validator — The Whole Glyph

- **Purpose** ⇒ Validates whether N is an integer
- **Regex** ⇒ Signed: `^([-]|[+])?[0-9]+$`; Unsigned: `^[0-9]+$`
- **Flag** B ⇒ If $B = 1$, allows optional sign; else requires unsigned digits
- **Use Case** ⇒ Numeric input validation for whole numbers

I Float Format Validator

__nx_is_float(N, B)

```
function __nx_is_float(N, B)
{
    return __nx_if(B, N ~ /^([-]|[+])?[0-9]+[.][0-9]+$/, N ~ /^[0-9]+[.][0-9]+$/)
}
```

Float Format Validator — The Decimal Glyph

- **Purpose** ⇒ Validates whether N is a decimal number
- **Regex** ⇒ Signed: `^([-]|[+])?[0-9]+[.][0-9]+$`; Unsigned: `^[0-9]+[.][0-9]+$`
- **Flag** B ⇒ If $B = 1$, allows optional sign; else requires unsigned format
- **Use Case** ⇒ Numeric input validation for floating-point values

I Natural Number Filter

nx_natural(N, B)

```
function nx_natural(N, B)
{
    if (__nx_is_integral(N, B) && +N > 0)
        return +N
}
```

Natural Number Filter — The Positive Glyph

- **Purpose** ⇒ Returns N if it's a valid positive integer
- **Validation** ⇒ Delegates to `__nx_is_integral(N, B)`
- **Condition** ⇒ Requires $N > 0$
- **Use Case** ⇒ Filtering for natural numbers in input streams

I Non-Zero Digit Filter

nx_not_zero(N, B)

```
function nx_not_zero(N, B)
{
    if (nx_digit(N, B) && +N != 0)
        return +N
}
```

Non-Zero Digit Filter — The Zero Guard Glyph

- **Purpose** ⇒ Returns N if it's a valid digit and not zero
- **Validation** ⇒ Delegates to `nx_digit(N, B)`
- **Condition** ⇒ Requires $N \neq 0$
- **Use Case** ⇒ Zero-guarded numeric filtering for input validation

I Numeric Format Validator

nx_digit(N, B1, B2)

```
function nx_digit(N, B1, B2)
{
    if (__nx_is_integral(N, B1) || __nx_is_float(N, __nx_else(B2,
↪B1)))
        return +N
}
```

Numeric Format Validator — The Composite Glyph

- **Purpose** ⇒ Returns N if it's a valid integer or float based on flags
- **Validation** ⇒ Checks __nx_is_integral(N, B1) or __nx_is_float(N, __nx_else(B2, B1)))
- **Flags** ⇒ $B1$ controls integer sign allowance; $B2$ controls float policy, defaulting to $B1$
- **Return** ⇒ Casts string N to numeric with +N
- **Use Case** ⇒ Generic numeric input filtering where decimals may be permitted

I Precision Formatter

__nx_precision(N1, N2)

```
function __nx_precision(N1, N2)
{
    if (nx_digit(N1, 1)) {
        N1 = sprintf("%." __nx_if(__nx_is_integral(N2), N2, "")
↪"f", N1)
        if (__nx_is_float(N1, 1))
            gsub(/(00+$)/, "", N1)
        return N1
    }
}
```

Precision Formatter — The Precision Glyph

- **Purpose** ⇒ Validates and formats numeric string $N1$ with decimal precision $N2$
- **Precision Parameter** ⇒ Uses __nx_if(__nx_is_integral(N2), N2, "") to set decimal digits
- **Format Assembly** ⇒ Builds format string "%<precision>f" for `sprintf`
- **Trailing Zero Removal** ⇒ Strips redundant zeros via `gsub(/(00+$)/, "", N1)`
- **Return** ⇒ Returns formatted string or undefined if $N1$ fails `nx_digit(N1,1)`

I Digit Guard Filter

nx_digit_guard(N, B1, B2, S, i, v)

```
function nx_digit_guard(N, B1, B2, S,   i, v)
{
    nx_trim_split(N, v, S)
    B2 = int(B2)
    B1 = int(B1)
    v["-1"] = __nx_if(B1, "optional signed", "required unsigned")
    v["-2"] = __nx_if(B2, "optional signed", "required unsigned")
    i = 0
    do {
        if (B2 == 2 && ! __nx_is_integral(v[v[0]], B1)) {
            nx_log_stderr(nx_log_error(v[v[0]] " is not a valid "
↪v["-1"] " integral!"))
            i = -1
        } else if (B2 == 3 && ! __nx_is_float(v[v[0]], B1)) {
            nx_log_stderr(nx_log_error(v[v[0]] " is not a valid "
↪v["-1"] " floating point digit!"))
            i = -1
        } else if (! nx_digit(v[v[0]], B1, B2)) {
            nx_log_stderr(nx_log_error(v[v[0]] " is not a valid "
↪v["-1"] " integral, nor is it a valid " v["-2"] " floating point
↪digit!"))
            i = -1
        }
    } while (--v[0] > 0)
    delete v
    return i
}
```

Digit Guard Filter — The Guardian Glyph

- **Purpose** ⇒ Validates a delimited list of numeric tokens in string N based on type policy
- **Splitting** ⇒ Trims and splits N by separator S into array v via `nx_trim_split(N, v, S)`
- **Flags** ⇒ $B1$: sign allowance flag; $B2$: numeric type (2=integral, 3=float, other=generic)
- **Validation** ⇒ Iterates tokens $v[1\ldots]$, enforces type via __nx_is_integral, __nx_is_float, or nx_digit
- **Error Handling** ⇒ Logs first invalid token with `nx_log_stderr(nx_log_error(...))` and returns -1; else returns 0

II Json Module

II Using the JSON value splitter

Json used in the following examples

```
{
  "user": {
    "id": 42,
    "name": "Alice",
    "email": "alice@example.com",
    "roles": [ "admin", "editor", "viewer" ]
  },
  "settings": {
    "theme": "dark",
    "notifications": true,
    "languages": [ "en", "fr" ]
  },
  "projects": [
    { "title": "Apollo", "status": "active" },
    { "title": "Zephyr", "status": "archived" }
  ],
  "misc": null
}
```

Extract keys from the `.user` object

Call ⇝ `nx_json_split(".user", V1, V2, "")`

Return ⇝ 1 (object)

Result ⇝ V2[0]=4, keys = {"id","name","email","roles"}

```
if (nx_json_split(".user", V1, V2, "") == 1) {
    for (i = 1; i <= V2[0]; i++) {
        key = V2[i]
        val = V1[".nx.user." key]
        print key, "=", val
    }
}
```

Extract values from the `.user.roles` array

Call ⇝ `nx_json_split(".user.roles", V1, V2, "")`

Return ⇝ 2 (array)

Result ⇝ V2[0]=3, values = {"admin","editor","viewer"}

```
if (nx_json_split(".user.roles", V1, V2, "") == 2) {
    for (i = 1; i <= V2[0]; i++)
        print V2[i]
}
```

Extract project entries from the `.projects` array

Call ⇝ `nx_json_split(".projects", V1, V2, "")`

Return ⇝ 2 (array)

Result ⇝ V2[0]=2, entries = {".nx.projects[1]",".nx.projects[2]"}

```
if (nx_json_split(".projects", V1, V2, "") == 2) {
    for (i = 1; i <= V2[0]; i++)
        print V2[i]
}
```

Attempt to split the `.misc` field (null)

- Call ⇝ `nx_json_split(".misc", V1, V2, "")`
- Return ⇝ 0 (no split)
- Result ⇝ V2 untouched; safe to check return code before iterating

```
if (nx_json_split(".misc", V1, V2, "") == 0)
    print "misc is null or undefined"
```

III Logging Module

III Signed Number Detection

nx_fd_stderr(D)

```
function nx_fd_stderr(D)
{
        if (D) {
                gsub(/"/, "\\\"" D)
                system("printf \"" D "\" 1>&2")
        }
}
```

- **Purpose** ⇝ Emit string D to standard error using shell printf
- **Escaping** ⇝ Double quotes in D are escaped via `gsub`
- **Condition** ⇝ Only executes if D is non-empty
- **Shell Invocation** ⇝ Uses `system("printf ...")` for stderr emission
- **Edge Case** ⇝ Newlines and shell metacharacters are passed verbatim—input must be trusted
- **Compliance** ⇝ POSIX AWK compatible; no refactor required
- **Use Case** ⇝ Diagnostic logging, error propagation, or stderr tracing in AWK scripts

III ANSI Style Mapper

__nx_ansi_smap(D)

```
function __nx_ansi_smap(D,  c)
{
    if (D == "o") # overline
        return 53
    if (D == "O") # not overline
        return 55
    if (D != (c = tolower(D))) {
        D = c
        c = 20
    } else {
        c = 0
    }
    if (D == "n") # normal
        return "0"
    if (D == "b") # bold
        return c + 1
    if (D == "d") # dim
        return c + 2
    if (D == "i") # italic
        return c + 3
    if (D == "u") # underline
        return c + 4
    if (D == "f") # flash
        return c + 5
    if (D == "r") # reverse video
        return c + 7
    if (D == "h") # hide
        return c + 8
    if (D == "s") # strike
        return c + 9
    return 0
}
```

ANSI Style Mapper — The Style Glyph

- **Purpose** ⇝ Maps style code D to ANSI numeric value
- **Input** ⇝ Single-character code D; case-sensitive for overline variants
- **Brightness** ⇝ Uppercase D triggers bright variant (adds 20)
- **Special Codes** ⇝ o=overline (53), O=not overline (55)
- **Use Case** ⇝ Style formatting for terminal output or ANSI escape generation

ANSI Style Code Map

Code	Mapped ANSI Value
n	Normal (0)
b	Bold (1) / Bright Bold (21)
d	Dim (2) / Bright Dim (22)
i	Italic (3) / Bright Italic (23)
u	Underline (4) / Bright Underline (24)
f	Flash (5) / Bright Flash (25)
r	Reverse Video (7) / Bright Reverse (27)
h	Hide (8) / Bright Hide (28)
s	Strike (9) / Bright Strike (29)
o	Overline (53)
O	Not Overline (55)
Add 20 for bright variant (uppercase *D*)	

III ANSI Color Mapper

__nx_ansi_cmap(D1, D2)

```
function __nx_ansi_cmap(D1, D2,     c)
{
    if (D2 == "<")
        c = 10
    else
        c = 0
    if ((D2 = tolower(D1)) == "c")
        return c + 39
    if (D2 == "r")
        return c + 38
    if (D1 != D2)
        c += 60
    if (D2 == "b")
        return c + 30
    if (D2 == "e")
        return c + 31
    if (D2 == "s")
        return c + 32
    if (D2 == "w")
        return c + 33
    if (D2 == "i")
        return c + 34
```

```
        if (D2 == "d")
            return c + 35
        if (D2 == "a")
            return c + 36
        if (D2 == "l")
            return c + 37
        return 0
    }
```

ANSI Color Mapper — The Chromatic Glyph

- **Purpose** ⇝ Maps color code $D1$ to ANSI numeric value
- **Foreground** ⇝ Use > in $D2$ to select foreground mode
- **Background** ⇝ Use < in $D2$ to select background mode
- **Brightness** ⇝ Uppercase $D1$ triggers bright variant (adds 60)
- **Normalization** ⇝ Lowercase $D1$ yields normal intensity
- **Special Codes** ⇝ `c`=default (39), `r`=reset (38)
- **Use Case** ⇝ Color formatting for terminal output or ANSI escape generation

ANSI Color Code Map

Code	Base ANSI Value
b	Black (30) / Bright Black (90)
e	Red (31) / Bright Red (91)
s	Green (32) / Bright Green (92)
w	Yellow (33) / Bright Yellow (93)
i	Blue (34) / Bright Blue (94)
d	Magenta (35) / Bright Magenta (95)
a	Cyan (36) / Bright Cyan (96)
l	Light gray (37) / Bright White (97)
c	Default color (39)
r	Reset foreground (38)
Add 10 for background mode (<)	
Add 60 for bright variant (uppercase $D1$)	

III ANSI Print Emitter

nx_ansi_print(D, B)

```
function nx_ansi_print(D, B,    stk, ansi, args)
{
    # D has no more use, lets store the index counter in it
    if ((D = split(D, args, "<nx:null/>")) < 2) {
        delete args
        return -1
    }
    args[1] = split("2" args[1], ansi, "")
    ansi[0] = args[1]

    args[0] = D # varargs count
    args[1] = 2 # varargs index
    D = ""

    stk[0] = 2
    stk[1] = ">"
    stk[2] = "\0"
    stk[3] = "<nx:false/>"
    do {
        if (ansi[ansi[1]] ~ /[<>_\0]/) {
            stk[1] = ansi[ansi[1]]
        } else if (ansi[ansi[1]] == "^") {
```

```
23              stk[2] = __nx_ansi_pmap(ansi[++ansi[1]])
24          } else if (ansi[ansi[1]] == "%") {
25              D = D __nx_if(stk[3] == "<nx:false/>", "", "m")
    ↪__nx_if(stk[1] == "\0" || stk[2] == "\0", "", "[" stk[2]
    ↪"]\x1b[37m: ") __nx_if(stk[4], stk[4] "m", "") args[args[1]++]
26              stk[3] = "<nx:false/>"
27              stk[4] = ""
28          } else {
29              if (stk[1] ~ /[<>_]/) {
30                  stk[5] = __nx_if(stk[3] == "<nx:false/>", "\x1b[",
    ↪";") __nx_if(stk[1] == "_", __nx_ansi_smap(ansi[ansi[1]]),
    ↪__nx_ansi_cmap(ansi[ansi[1]], stk[1]))
31                  D = D stk[5]
32                  stk[4] = stk[4] stk[5]
33                  stk[3] = "<nx:true/>"
34              } else {
35                  D = D __nx_if(stk[3] == "<nx:false/>", "", "m")
    ↪ansi[ansi[1]]
36                  stk[3] = "<nx:false/>"
37              }
38          }
39      } while (args[1] <= args[0] && ++ansi[1] <= ansi[0])
40      if (B != "<nx:true/>") {
41          nx_fd_stderr(D "\x1b[0m")
42          D = args[1]
43      }
44      delete args
45      delete stk
46      delete ansi
47      return D
48  }
```

ANSI Print Emitter — The Render Glyph

- **Purpose** ⇝ Renders styled output using ANSI sequences and symbolic glyphs
- **Input** ⇝ D: markup string with embedded style codes and placeholders; B: optional suppress flag
- **Markup** ⇝ Uses `<nx:null/>` as delimiter for varargs; style codes include <, >, _
- **Glyphs** ⇝ Symbolic glyphs injected via ^ and mapped by `__nx_ansi_pmap`
- **Substitution** ⇝ % triggers vararg injection with optional prefix and style context
- **Style Codes** ⇝ Resolved via `__nx_ansi_smap` or `__nx_ansi_cmap` depending on mode
- **Output** ⇝ Emitted to stderr unless $B =$ `<nx:true/>`; resets with `\x1b[0m`
- **Use Case** ⇝ Diagnostic rendering, styled logging, or symbolic output in AWK scripts

III ANSI Pictographic Mapper

__nx_ansi_pmap(D)

```
function __nx_ansi_pmap(D)
{
    if (D == "b") { # emphasis
        D = "#"
    } else if (D == "B") { # pipeline
        D = "|"
    } else if (D == "e") { # minor error
        D = "x"
    } else if (D == "E") { # critical error needs attention like
↪yesterday
        D = "X"
    } else if (D == "s") { # success
        D = "v"
    } else if (D == "S") { # great success
        D = "V"
    } else if (D == "w") { # warning
        D = "!"
    } else if (D == "W") { # warning but not sure
        D = "?"
    } else if (D == "d") { # debug
        D = "*"
    } else if (D == "D") { # trace
        D = ">"
    } else if (D == "i") { # info
        D = "i"
```

```
25      } else if (D == "I") { # verbose
26          D = "."
27      } else if (D == "l") { # log
28          D = "%"
29      } else if (D == "L") { # detailed log
30          D = "$"
31      } else if (D == "a") { # alert
32          D = "@"
33      } else if (D == "A") { # more info alert
34          D = "&"
35      } else {
36          return "\0"
37      }
38      return nx_ansi_print("_uir%_UIR%<nx:null/>" D "<nx:null/>",
    ↪"<nx:true/>")
39  }
```

ANSI Pictographic Mapper — The Symbol Glyph

- **Purpose** ⇝ Maps symbolic code D to a printable glyph
- **Input** ⇝ Single-character code D; case-sensitive
- **Fallback** ⇝ Returns `"\0"` if code is unrecognized
- **Rendering** ⇝ Delegates to `nx_ansi_print(...)` for styled output
- **Use Case** ⇝ Diagnostic symbols, log annotations, or status glyphs

ANSI Symbol Code Map

Code	Mapped Glyph
b	# (emphasis)
B	| (pipeline)
e	x (minor error)
E	X (critical error)
s	v (success)
S	V (great success)
w	! (warning)
W	? (uncertain warning)
d	* (debug)
D	> (trace)
i	i (info)
I	. (verbose)
l	% (log)
L	$ (detailed log)
a	@ (alert)
A	& (info alert)

IV Miscellaneous Module

IV Character Classifiers

nx_is_space(D)

```
function nx_is_space(D) { return D ~ /[ \t\n\f\r\v\b]/ }
```

nx_is_upper(D)

```
function nx_is_upper(D) { return D ~ /[A-Z]/ }
```

nx_is_lower(D)

```
function nx_is_lower(D) { return D ~ /[a-z]/ }
```

nx_is_alpha(D)

```
function nx_is_alpha(D) { return nx_is_lower(D) || nx_is_upper(D) }
```

nx_is_digit(D)

```
function nx_is_digit(D) { return D ~ /[0-9]/ }
```

Character Classifiers — The Glyph Filters

- nx_is_space ⇝ Checks for whitespace characters
- nx_is_upper ⇝ Checks for uppercase letters
- nx_is_lower ⇝ Checks for lowercase letters
- nx_is_alpha ⇝ Checks for alphabetic characters
- nx_is_digit ⇝ Checks for numeric digits

IV File Presence Checker

nx_is_file(D)

```
function nx_is_file(D)
{
        if ((getline < D) > 0)
                close(D)
        else
                return 0
        return 1
}
```

File Presence Checker — The Path Glyph

- **Purpose** ⇝ Check if file D exists and is readable
- **Mechanism** ⇝ Attempts `getline`; closes if successful
- **Return** ⇝ 1 if file is readable, 0 otherwise
- **Use Case** ⇝ File validation, path probing, or conditional loading

IV Presence Evaluator

__nx_defined(D, B)

```
function __nx_defined(D, B)
{
                return (D || (length(D) && B))
}
```

Presence Evaluator — The Defined Glyph

- **Purpose** ⇝ Determines whether value D is considered present or truthy
- **Input** ⇝ D: value to check; B: fallback flag
- **Logic** ⇝ Returns true if D is non-empty or if $length(D)$ and B are both true
- **Use Case** ⇝ Used by conditional glyphs to evaluate presence, fallback, or symbolic truth

IV Fallback Selector

__nx_else(D1, D2, B)

```
function __nx_else(D1, D2, B)
{
        if (D1 || __nx_defined(D1, B))
                return D1
        return D2
}
```

Fallback Selector — The Else Glyph

- **Purpose** ⇝ Returns $D1$ if defined or truthy, otherwise returns fallback $D2$
- **Input** ⇝ $D1$: primary value; $D2$: fallback value; B: optional presence flag
- **Logic** ⇝ Uses `__nx_defined(D1, B)` to determine presence
- **Use Case** ⇝ Used in conditional chains, defaulting logic, or symbolic substitution

IV Conditional Selector

__nx_if(B1, D1, D2, B2)

```
function __nx_if(B1, D1, D2, B2)
{
        if (B1 || __nx_defined(B1, B2))
                return D1
        return D2
}
```

Conditional Selector — The If Glyph

- **Purpose** ⇝ Returns $D1$ if condition $B1$ is true or defined, otherwise returns fallback $D2$
- **Input** ⇝ $B1$: primary condition; $D1$: value if true; $D2$: value if false; $B2$: optional presence flag
- **Logic** ⇝ Uses `__nx_defined(B1, B2)` to evaluate symbolic truth
- **Use Case** ⇝ Conditional rendering, symbolic branching, or fallback substitution in AWK emitters

IV Ternary Divergence

__nx_elif(B1, B2, B3, B4, B5, B6)

```
function __nx_elif(B1, B2, B3, B4, B5, B6)
{
    if (B4) {
        B5 = __nx_else(B5, B4)
        B6 = __nx_else(B6, B5)
    }
    return (__nx_defined(B1, B4) == __nx_defined(B2, B5) &&
            __nx_defined(B3, B6) != __nx_defined(B1, B4))
}
```

Ternary Divergence — The Elif Glyph

- **Purpose** ⇝ Evaluates a three-way conditional divergence based on symbolic presence and equality
- **Input** ⇝ $B1, B2, B3$: primary conditions; $B4, B5, B6$: optional fallback values
- **Fallback** ⇝ Each fallback is resolved via `__nx_else` to normalize symbolic presence
- **Logic** ⇝ Returns true if $\texttt{defined}(B1) = \texttt{defined}(B2)$ and $\texttt{defined}(B3) \neq \texttt{defined}(B1)$
- **Use Case** ⇝ Used in chained conditional branches where symbolic presence and divergence must be tested

IV Conditional Union

__nx_or(B1, B2, B3, B4, B5, B6)

```
function __nx_or(B1, B2, B3, B4, B5, B6)
{
    if (B4) {
        B5 = __nx_else(B5, B4)
        B6 = __nx_else(B6, B5)
    }
    return ((__nx_defined(B1, B4) && __nx_defined(B2, B5)) ||
            (__nx_defined(B3, B6) && ! __nx_defined(B1, B4)))
}
```

Conditional Union — The Or Glyph

- **Purpose** ⇝ Evaluates symbolic union of conditions with fallback normalization
- **Input** ⇝ $B1$, $B2$, $B3$: primary conditions; $B4$, $B5$, $B6$: optional fallback values
- **Fallback** ⇝ Each fallback is resolved via `__nx_else` to normalize symbolic presence
- **Logic** ⇝ Returns true if `defined(B1) and defined(B2) or defined(B3) and not defined(B1)`
- **Use Case** ⇝ Used in conditional branching, symbolic union, or fallback-aware logic overlays

Logical OR with both primary values defined

- **Inputs** ⇝ Left: `"yes"`, Right: `"ok"`
- **Operation** ⇝ Both values are defined
- **Expected Result** ⇝ Returns `true`

```
__nx_or("yes", "ok", "", "", "", "")
```

Logical OR with only fallback defined

- **Inputs** ⇝ Fallback: `"fallback"`, Primaries: `""`
- **Operation** ⇝ Evaluates fallback since primaries are undefined
- **Expected Result** ⇝ Returns `true`

```
__nx_or("", "", "fallback", "", "", "")
```

Logical OR with all values undefined

- **Inputs** ⇝ All arguments empty
- **Operation** ⇝ No defined values to satisfy OR condition
- **Expected Result** ⇝ Returns `false`

```
__nx_or("", "", "", "", "", "")
```

IV Exclusive Divergence

__nx_xor(B1, B2, B3, B4)

```
function __nx_xor(B1, B2, B3, B4)
{
    if (B3)
        B4 = __nx_else(B4, B3)
    return ((! __nx_defined(B2, B4) && __nx_defined(B1, B3)) ||
            (__nx_defined(B2, B4) && ! __nx_defined(B1, B3)))
}
```

Exclusive Divergence — The Xor Glyph

- **Purpose** ⇝ Returns true if exactly one of the two symbolic conditions is defined or truthy
- **Input** ⇝ $B1$, $B2$: primary conditions; $B3$, $B4$: optional fallback values
- **Fallback** ⇝ $B4$ is resolved via `__nx_else(B4, B3)`
- **Logic** ⇝ Returns true if one is defined and the other is not
- **Use Case** ⇝ Used in symbolic branching, exclusive logic, or markup divergence testing

Exclusive-or with only the first value defined

- Inputs ⇝ Left: `"yes"`, Right: `""`
- Operation ⇝ One side defined, the other undefined
- Expected Result ⇝ Returns `true`

```
__nx_xor("yes", "", "", "")
```

Exclusive-or with both values defined

- Inputs ⇝ Left: `"yes"`, Right: `"ok"`
- Operation ⇝ Both sides defined
- Expected Result ⇝ Returns `false`

```
__nx_xor("yes", "ok", "", "")
```

Exclusive-or with only the fallback defined

- Inputs ⇝ Fallback: `"fallback"`, Others: `""`
- Operation ⇝ One side defined via fallback, the other undefined
- Expected Result ⇝ Returns `true`

```
__nx_xor("", "", "fallback", "")
```

Exclusive-or with only the first value defined

- Inputs ⇝ Left: `"yes"`, Right: `""`
- Operation ⇝ One side defined, the other undefined
- Expected Result ⇝ Returns `true`

```
__nx_xor("yes", "", "", "")
```

IV Symbolic Comparator

__nx_compare(B1, B2, B3, B4)

```
function __nx_compare(B1, B2, B3, B4)
{
    if (! B3) {
        if (length(B3)) {
            B1 = length(B1)
            B2 = length(B2)
        } else if (nx_digit(B1, 1) && nx_digit(B2, 1)) {
            B1 = +B1
            B2 = +B2
        } else {
            B1 = "a" B1
            B2 = "a" B2
        }
        B3 = 1
    }
    if (B4)
        return __nx_if(nx_digit(B4), B1 > B2, B1 < B2) ||
               __nx_if(__nx_else(nx_digit(B4) == 1, tolower(B4) ==
↪"i"), B1 == B2, 0)
    if (length(B4))
        return B1 ~ B2
    return B1 == B2
}
```

Symbolic Comparator — The Compare Glyph

- **Purpose** ⇝ Compares two values $B1$ and $B2$ using symbolic or numeric logic
- **Input** ⇝ $B1$, $B2$: values to compare; $B3$: mode flag; $B4$: comparison operator
- **Mode** ⇝ If $B3$ is empty, auto-selects: length, numeric, or string coercion
- **Operator** ⇝ If $B4$ is set: `digit` → numeric compare, `i` → equality, else regex match
- **Use Case** ⇝ Used in symbolic evaluation, numeric comparison, or pattern matching logic

Compare two numeric strings using digit-based operator

- **Inputs** ⇝ Left: `"5"`, Right: `"10"`, Mode: `""`, Operator: `"1"`
- **Operation** ⇝ Numeric comparison with coercion via digit detection
- **Expected Result** ⇝ Returns `true` since `5 < 10`

```
__nx_compare("5", "10", "", "1")
```

Match string against pattern using regex operator

- **Inputs** ⇝ Left: `"abc"`, Operator: `" "`, Right: `"b"`
- **Mode** ⇝ Regex match against pattern `"b"`
- **Expected Result** ⇝ Returns `true` since `"abc"` contains `"b"`

```
__nx_equality("abc", "~", "b")
```

Compare two strings using alphabetic equality operator

- **Inputs** ⇝ Left: `"abc"`, Operator: `"=a"`, Right: `"abc"`
- **Mode** ⇝ Alphabetic equality check
- **Expected Result** ⇝ Returns `true` since both strings are equal

```
__nx_equality("abc", "=a", "abc")
```

IV Symbolic Equality Evaluator

__nx_equality(B1, B2, B3)

```
function __nx_equality(B1, B2, B3,  b, e, g)
{
    b = substr(B2, 1, 1)
    if (b == ">") {
```

```
            e = 2
            g = 1
        } else if (b == "<") {
            e = "a"
            g = "i"
        } else if (b == "=") {
            e = ""
        } else if (b == "~") {
            e = 0
        } else {
            b = ""
        }
        if (b) {
            if (__nx_compare(substr(B2, 2, 1), "=", 1)) {
                b = g
            } else {
                b = e
            }
            e = substr(B2, length(B2), 1)
            if (__nx_compare(e, "a", 1))
                return __nx_compare(B1, B3, "", b)
            else if (__nx_compare(e, "_", 1))
                return __nx_compare(B1, B3, 0, b)
            else
                return __nx_compare(B1, B3, 1, b)
        }
        return __nx_compare(B1, B2)
    }
```

Symbolic Equality Evaluator — The Equality Glyph

- **Purpose** ⇝ Evaluates symbolic equality or comparison between $B1$ and $B3$ using operator $B2$
- **Input** ⇝ $B1$: left value; $B2$: operator string; $B3$: right value
- **Operators** ⇝ Supports >, <, =, with optional suffixes (a, _, etc.)
- **Logic** ⇝ Delegates to `__nx_compare` with inferred mode and operator
- **Use Case** ⇝ Used in symbolic evaluation, conditional logic, or markup comparison overlays

Compare two numeric strings using digit-based operator

- **Inputs** ⇝ Left: `"5"`, Operator: `">0"`, Right: `"3"`
- **Mode** ⇝ Numeric comparison using digit coercion
- **Expected Result** ⇝ Returns `true` since `5 > 3`

```
__nx_equality("5", ">0", "3")
```

Match string against pattern using regex operator

- **Inputs** ⇝ Left: `"abc"`, Operator: `"~"`, Right: `"b"`
- **Mode** ⇝ Regex match against pattern `"b"`
- **Expected Result** ⇝ Returns `true` since `"abc"` contains `"b"`

```
__nx_equality("abc", "~", "b")
```

Compare two strings using alphabetic equality operator

- **Inputs** ⇝ Left: `"abc"`, Operator: `"=a"`, Right: `"abc"`
- **Mode** ⇝ Alphabetic equality check
- **Expected Result** ⇝ Returns `true` since both strings are equal

```
__nx_equality("abc", "=a", "abc")
```

V String Module

V String Reaper

String Reaper Arguments

- D1 ⇝ Input string to modify
- N1 ⇝ Count of removals (integer, absolute)
- D2 ⇝ Substring/pattern to remove
- N2 ⇝ Direction flag:
  - 0 ⇝ Remove forward (left to right)
  - 1 ⇝ Remove backward (right to left)

```
function nx_reap_str(D1, N1, D2, N2)
```

String Reaper — Controlled Substring Removal

- **Purpose** ⇝ Removes occurrences of substring $D2$ from string $D1$, with count and direction control
- **Input** ⇝ $D1$: target string; $N1$: number of removals (integer, absolute); $D2$: substring to remove; $N2$: direction flag (0 = forward, 1 = backward)
- **Mechanism** ⇝ If $N1$ is invalid or zero, all matches of $D2$ are removed. Otherwise, removes up to $N1$ matches, scanning left-to-right if $N2 = 0$ or right-to-left if $N2 = 1$
- **Return** ⇝ Modified string with requested removals applied
- **Use Case** ⇝ Used in keyword argument pop operations, controlled string mutation, or cleanup of repeated tokens

V Examples

Remove first occurrence (forward pop)

- Input ⇝ ‘hello␣world␣hello’
- N1 ⇝ 1
- Substring ⇝ ‘hello’
- Direction ⇝ Forward (0)
- Expected Result ⇝ “␣world␣hello”

```
print nx_reap_str("hello world hello", 1, "hello", 0)
```

Remove last occurrence (backward pop)

- Input ⇝ “hello world hello”
- N1 ⇝ 1
- Substring ⇝ “hello”
- Direction ⇝ Backward (1)
- Expected Result ⇝ “hello␣world␣”

```
print nx_reap_str("hello world hello", 1, "hello", 1)
```

Remove all occurrences (zero count)

- Input ⇝ “foo␣bar␣foo␣bar”
- N1 ⇝ 0
- Substring ⇝ “foo”
- Direction ⇝ Forward (0)
- Expected Result ⇝ “␣bar␣bar”

```
print nx_reap_str("foo bar foo bar", 0, "foo", 0)
```

VI Struct Module

VI Bijective Mapper

nx_bijective Arguments

- V ⟿ Associative array (registry) to mutate
- D1 ⟿ Primary key (source of mapping)
- D2 ⟿ Secondary key (target or symmetric partner)
- D3 ⟿ Tertiary key (cycle target or reassignment value)

```
function nx_bijective(V, D1, D2, D3)
```

Bijective Mapper — The Mapping Glyph

- **Purpose** ⟿ Establishes or mutates bijective relationships among symbolic keys in V
- **Input** ⟿ V: registry; $D1$, $D2$, $D3$: keys to bind in symmetric or cyclic form
- **Mechanism**
  - Three arguments $\rightarrow$ create 3-cycle $D1 \rightarrow D2 \rightarrow D3 \rightarrow D1$.
  - Two arguments $\rightarrow$ create symmetric pair $D1 \leftrightarrow D2$.
  - One reassignment argument $\rightarrow$ redirect existing mapping of $D1$ to $D3$.
- **Return** ⟿ No explicit return; modifies V in place
- **Use Case** ⟿ Used in overlay systems to guarantee reversible, navigable mappings between symbolic identifiers

VI Examples

Create symmetric pair mapping

- **Input** → D1 = ‘a’, D2 = ‘b’
- **Operation** → Sets V[‘a’] = ‘b’ and V[‘b’] = ‘a’
- **Expected Result** → a ↔ b bijection established

```
nx_bijective(V, "a", "b")
```

Create cyclic triple mapping

- **Input** → D1 = ‘x’, D2 = ‘y’, D3 = ‘z’
- **Operation** → Sets x → y, y → z, z → x
- **Expected Result** → Cycle established among three keys

```
nx_bijective(V, "x", "y", "z")
```

Mutate existing mapping

- **Initial State** → V[‘a’] = ‘b’, V[‘b’] = ‘a’
- **Input** → D1 = ‘a’, D2 = “”, D3 = ‘c’
- **Operation** → Reassigns V[‘b’] = ‘c’, optionally deletes V[‘a’]
- **Expected Result** → Mapping updated: b → c

```
nx_bijective(V, "a", "", "c")
```

VI Grid Queue Conductor

nx_grid Arguments

- V — Associative array acting as the grid/queue registry
- D — Data element to insert (string); empty when retrieving
- N — Control index: row selector, retrieval mode, or deletion flag

```
function nx_grid(V, D, N)
```

Grid Queue Conductor — The Grid Glyph

- **Purpose** — Implements a sparse grid/queue structure inside associative array V
- **Input** — V: registry; D: element to insert; N: row index or control directive
- **Mechanism**
  - Initializes grid on first use
  - Appends new entries to next available row/column
  - Retrieves or deletes entries depending on N and D.
- **Indexes**
  - **[0]** — highest allocated row
  - **["-"]** — lowest active row (head pointer)
  - **["|"]** — current column pointer within active row
- **Return** — When retrieving, returns the element at the current grid position
- **Use Case** — Queueing, scheduling, IPC overlays, or grid-like storage in AWK daemons

VI Examples

Insert element into grid at next slot

- Initial State → Empty grid
- Operation → Insert “alpha”
- Expected Result → Stored at 1 , 1; grid initialized with [0]=1, [“-”]=1, [“|”]=1

```
nx_grid(V, "alpha")
```

Retrieve last inserted element

- State → Grid contains “alpha” at 1 , 1
- Operation → Call with N=1
- Expected Result → Returns “alpha”

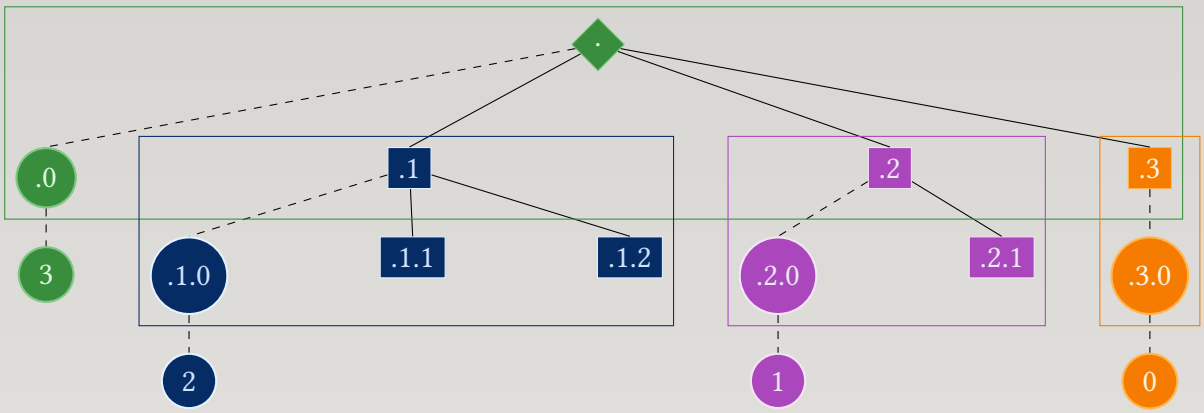
```
print nx_grid(V, "", 1)
```

Iterate forward through grid

- State → Grid contains multiple entries
- Operation → Call with no D, no N
- Expected Result → Returns next element at current [“-”], [“|”] position, advancing column pointer

```
print nx_grid(V)
```

VI Tree–Path Model



Node Path Forms in the DFS Tree

- **Root Prefix (‘.’)** ⟿ Implicit root path; never stored directly. Used as prefix for all top-level nodes.
- **Root Child Count (‘.0’)** ⟿ Stores the number of children under the root. Drives the initial DFS descent.
- **First Child Node (‘.1’)** ⟿ First child of the root. Addressed by appending index to root prefix.
- **Child Count of Node ‘.1’ (‘.1.0’)** ⟿ Stores how many children node ‘.1’ has. Enables recursive descent.
- **Second Child of Node ‘.1’ (‘.1.2’)** ⟿ Second child of node ‘.1’. Addressed by appending index.
- **Child Count of Node ‘.1.2’ (‘.1.2.0’)** ⟿ Stores how many children node ‘.1.2’ has. Continues the DFS scaffold.

Tree–Path Model — The Structural Glyph

- **Purpose** — Represent a full tree using only string paths and child counters, enabling single-pass, non-recursive DFS traversal
- **Root** — The root node is the empty path ‘’; its prefix is ‘.’, and its child count is stored at ‘.0’
- **Node Paths** — Each node is addressed by a dotted path: ‘.1’, ‘.2’, ‘.1.3’, ‘.1.3.2’, etc.
- **Child Counters** — Every node stores its child count at ‘<node>.0’, e.g., ‘.1.0’ or ‘.1.2.0’
- **Child Nodes** — Children are stored at ‘<node>.<index>’, e.g., ‘.1’, ‘.2’, ‘.1.1’, ‘.1.2’
- **Traversal** — The DFS engine walks the tree by incrementing child indices and descending whenever ‘<node>.0’ is non-zero
- **Stack Model** — A manual stack stores ‘(si, sj)’ pairs for each descent, enabling recursion-free traversal
- **Flattening** — `nx_dfs` rewrites the tree into ‘V[1..N]’ in depth-first order, with ‘V[0]’ holding the total count
- **Use Case** — Foundation for include-merging, file expansion, and any structure requiring ordered, hierarchical flattening

VI Examples

Simple Tree Structure

- **State** — `.0 = 2`, children at ‘.1’, ‘.2’
- **Operation** — Call `nx_dfs(V)`
- **Result** — `V[1] = '.1', V[2] = '.2'`

```
nx_dfs(V)
```

Nested Tree Structure

- State → `.0 = 1, .1.0 = 2`, children ‘.1.1’, ‘.1.2’
- Operation → Call `nx_dfs(V)`
- Result → `V[1] = ‘.1’, V[2] = ‘.1.1’, V[3] = ‘.1.2’`

```
nx_dfs(V)
```

VII I/O Modules

VII File Readability Sentinel

nx_is_file Arguments

- D → Candidate file path to probe
- B → Boolean or mode flag controlling strictness of the readability test

```
function nx_is_file(D, B)
```

File Readability Sentinel — The Probe Glyph

- **Purpose** — Determine whether a given path ‘D’ refers to a readable file, with strict or permissive content requirements depending on ‘B’
- **Input** — ‘D’ is the file to test; ‘B’ selects the probe threshold:
  - </> $B = ""$ — Strict mode: `getline < D` must return > **0**, meaning the file is readable *and contains at least one line of content*
  - </> $B \neq ""$ — Permissive mode: `getline < D` must return > **-1**, meaning the file is readable even if it is *empty*
- **Mechanism**
  - Empty path/If ‘D’ is empty, immediately return ‘0’
  - Probe/Attempt `getline < D` using the threshold determined by ‘B’
  - Close/If the probe succeeds, close the stream handle
  - Failure/If the probe fails the threshold, return ‘0’
  - Success/Otherwise return ‘1’
- **Output** — Returns ‘1’ if the file is readable under the chosen mode, ‘0’ otherwise
- **Use Case** — Foundational predicate for all file-identity and include-expansion rituals; ensures that only real, readable files (optionally non-empty) enter the semantic universe

VII Examples

Strict readability check (non-empty file required)

- State → D=‘`config.cfg`’, B=‘ ’ (strict mode)
- Operation → Probe succeeds only if ‘config.cfg’ is readable *and contains at least one line*
- Expected Result → Returns ‘1’ for non-empty readable files; ‘0’ for empty or unreadable files

```
nx_is_file("config.cfg", "")
```

Permissive readability check (empty file allowed)

- State → D=‘`empty.log`’, B=‘1’ (permissive mode)
- Operation → Probe succeeds if ‘empty.log’ is readable, even if it contains no content
- Expected Result → Returns ‘1’ for readable files regardless of content; ‘0’ only if unreadable

```
nx_is_file("empty.log", 1)
```

VII Path Normalization Conductor

nx_file_path Arguments

- D1 → Input path to normalize or dissect
- B → Boolean or mode flag selecting which component to return
- D2 → Directory separator (default “/”)
- V → Prefix-expansion map for ‘NX_*:/’, ‘~/’, and ‘-/’

```
function nx_file_path(D1, B, D2, V)
```

Path Normalization Conductor — The Dissection Glyph

- **Purpose** — Normalize a file path, expand prefix sigils, collapse redundant separators, and optionally extract basename or directory components depending on ‘B’
- **Input** — ‘D1’ is the raw path; ‘B’ selects the return mode; ‘D2’ is the directory separator; ‘V’ holds prefix expansions:
  - </> $B = ""$ — Return the *basename* (final component)
  - </> $B = 0$ — Return the *full normalized path*
  - </> $B \neq ""$ — Return the *directory path* (parent directory)
- **Mechanism**
  - Prefix expansion/If ‘D1’ begins with ‘-/’, ‘~/’, or ‘NX_*:/’, expand using ‘V’ (environment-derived prefix map)
  - Separator collapse/Reduce repeated separators ‘D2+’ to a single ‘D2’
  - Trailing cleanup/Remove trailing separators unless the path is root
  - Basename extraction/If $B = ""$, strip directory prefix and return final component
  - Directory extraction/If $B \neq ""$, strip basename and return parent directory
  - Full path/If $B = 0$, return the normalized path unchanged
- **Output** — A normalized path, basename, or directory path depending on ‘B’
- **Use Case** — Core utility for all file-identity and include-expansion rituals; ensures consistent path semantics across prefix expansions, directory sweeps, and file lookups

VII Examples

Extract basename (strict basename mode)

- State → D1=‘/usr/local/bin/awk’, B=‘’
- Operation → Return only the final component
- Expected Result → ‘awk’

```
nx_file_path("/usr/local/bin/awk", "", "/")
```

Return full normalized path

- State → D1=‘ /projects///nx’, B=0
- Operation → Expand prefix, collapse separators, return full path
- Expected Result → Normalized absolute path to ‘nx’

```
nx_file_path("~/projects///nx", 0, "/", V)
```

Extract directory path (parent directory)

- State → D1=‘NX_C:/config/main.cfg’, B=1
- Operation → Expand ‘NX_C:/’, strip basename, return directory
- Expected Result → Parent directory of ‘main.cfg’

```
nx_file_path("NX_C:/config/main.cfg", 1, "/", V)
```

VII Extension Sweep Conductor

nx_file_path_expand Arguments

- D1 ⟿ Primary prefix path used when constructing candidate files
- D2 ⟿ Secondary prefix path used for alternate construction
- D3 ⟿ Basename to sweep for extensions
- D4 ⟿ Extension-separator regex (default '[.]')
- D5 ⟿ Directory separator (default "/")
- V1 ⟿ Parallel-array registry for file identity triples
- V2 ⟿ Prefix-expansion map for 'NX_*:/', '~/', and '-/'
- B1 ⟿ Boolean/mode flag selecting sweep strategy
- B2 ⟿ Boolean/mode flag passed to `nx_is_file`

```
function nx_file_path_expand(D1, D2, D3, D4, D5, V1, V2, B1, B2)
```

Extension Sweep Conductor — The Expansion Glyph

- **Purpose** — Perform an extension-sweep over a basename ‘D3’, generating candidate paths by peeling extensions and testing each candidate with `nx_file_store`
- **Input** — ‘D1’ and ‘D2’ are prefix paths; ‘D3’ is the basename; ‘D4’ and ‘D5’ define separators; ‘V1’ and ‘V2’ support file resolution; ‘B1’ and ‘B2’ control sweep and readability modes:
  - *B1* — Sweep mode: determines whether extension-peeling is active
  - *B2* — Readability mode passed to `nx_is_file`
- **Mechanism**
  - Reverse scan/Reverse ‘D3’ to locate extension boundaries using ‘D4’
  - Peel extension/Each match shortens the basename and accumulates peeled extensions into ‘ext’
  - Construct candidates/For each peel, construct:
    - **Candidate 1** — ‘D1 ext D5 bs’
    - **Candidate 2** — ‘D2 ext D5 D3’
  - Probe/Call `nx_file_store` on each candidate; success halts the sweep
  - Termination/Stop when a readable file is found or no more extension boundaries remain
- **Output** — Returns ‘1’ if any candidate path resolves to a readable file; ‘-1’ otherwise
- **Use Case** — Used by `nx_file_store` when fallback mode requests extension-peeling; enables discovery of files such as ‘config.json’, ‘config.yaml’, ‘config.conf’ by sweeping suffixes of ‘config’

VII Examples

Sweep for alternate extensions

- **State** ⇝ Basename ‘config.json’, prefixes ‘D1=‘/etc/’’, ‘D2=‘/usr/local/etc/’’
- **Operation** ⇝ Peel ‘.json’, test ‘config’, then test ‘config.json’ under both prefixes
- **Expected Result** ⇝ Returns ‘1’ if any candidate resolves to a readable file

```
nx_file_path_expand("/etc/", "/usr/local/etc/", "config.json",
↪"[.]", "/", V1, V2, 1, "")
```

Multiple extension layers (e.g., ‘.tar.gz’)

- **State** ⇝ Basename ‘archive.tar.gz’
- **Operation** ⇝ Peel ‘.gz’, then ‘.tar’, constructing candidates at each stage
- **Expected Result** ⇝ First readable candidate halts the sweep and returns ‘1’

```
nx_file_path_expand("/data/", "/backup/", "archive.tar.gz", "[.]",
↪"/", V1, V2, 1, 1)
```

VII File Identity Conductor

nx_file_store Arguments

- V1 ⟿ Parallel-array registry storing file identity triples
- D1 ⟿ Input file name (basename or relative path)
- D2 ⟿ Optional prefix path to search under
- V2 ⟿ Prefix-expansion map for ‘NX_*:/’, ‘~/’, and ‘-/’
- D3 ⟿ Directory separator (default “/”)
- B1 ⟿ Boolean/mode flag selecting fallback strategy
- B2 ⟿ Boolean/mode flag passed to `nx_is_file`
- D4 ⟿ Extension-separator regex used by `nx_file_path_expand`

```
function nx_file_store(V1, D1, D2, V2, D3, B1, B2, D4)
```

File Identity Conductor — The Triple Glyph

- **Purpose** — Resolve a file path, verify readability, and register its identity triple (real path, directory path, basename) into a bijective parallel-array registry
- **Input** — 'D1' is the candidate file; 'D2' is an optional search prefix; 'V2' holds prefix expansions; 'B1' and 'B2' control fallback and readability modes:
  - *B1 = 0* — No fallback: only direct path resolution attempted
  - *B1 = 1* — Extension-sweep fallback via `nx_file_path_expand`
  - *B1 = 2* — Directory-sweep fallback across known directories in 'V1'
  - *B1 = 3* — Combined sweep: extension first, then directory
  - *B2* — Strict/permissive readability mode passed to `nx_is_file`
- **Mechanism**
  - Normalize paths/Compute:
    - **rlp** — Resolved lookup path: 'D2 + D1' normalized
    - **orlp** — Original path: 'D1' normalized
    - **bs** — Basename extracted from 'D1'
  - Direct probe/Test 'rlp' and 'orlp' with `nx_is_file` under mode 'B2'
  - Directory sweep/If 'B1 = 2', iterate directories stored in 'V1' to locate a readable file
  - Extension sweep/If 'B1 = 1' or '3', call `nx_file_path_expand` to peel extensions and test candidates
  - Identity triple/Once a readable file is found:
    - **Real Path** — Canonical resolved path
    - **Directory Path** — Parent directory
    - **Basename** — Final component of the file name
  - Registry insertion/Insert each component into 'V1' using `nx_bijective` and `nx_parr_stk`
  - Return code/'1' if newly registered, '0' if already known, '-1' on failure

Output ⟿ Returns ‘1’ for a new identity triple, ‘0’ if already present, ‘-1’ if no readable file could be resolved

Use Case ⟿ Foundation of all file-merge, overlay, and include-expansion systems; ensures every file entering the semantic universe has a stable, bijective identity triple

VII Examples

Direct resolution (no fallback)

State ⟿ D1=‘config’, D2=‘NX_C:/’, B1=0

Operation ⟿ Resolve ‘NX_C:/config’ and register its identity triple

Expected Result ⟿ ‘1’ if new, ‘0’ if already known

```
nx_file_store(V1, "config", "NX_C:/", V2, "/", 0, "", "[.]")
```

Extension-sweep fallback

State ⟿ D1=‘theme’, B1=1

Operation ⟿ Try ‘theme’, then ‘theme.*’ by peeling extensions

Expected Result ⟿ First readable candidate registered

```
nx_file_store(V1, "theme", "", V2, "/", 1, "", "[.]")
```

Directory-sweep fallback

State ⟿ D1=‘settings.cfg’, B1=2

Operation ⟿ Search directories already registered in ‘V1’

Expected Result ⟿ First readable match registered

```
nx_file_store(V1, "settings.cfg", "", V2, "/", 2, "", "[.]")
```

VII Include Directive Conductor

nx_file_merge Arguments

- D1 — Root file name or input stream to expand
- D2 — File-exclusion list; omit these if encountered after a directive
- D3 — Separator used to split the delimiter specification ‘D4’
- D4 — Five-field delimiter specification:
  - [1] — File-exclusion separator
  - [2] — Directive sigil
  - [3] — Directory separator
  - [4] — File-extension separator
  - [5] — Directive name
- B — Boolean/mode flag passed to `nx_file_store`
- N — Verbosity and diagnostic level

```
function nx_file_merge(D1, D2, D3, D4, B, N)
```

Include Directive Conductor — The Expansion Glyph

- **Purpose** — Traverse a root file, detect include directives, recursively load referenced files, and emit a fully expanded DFS-ordered text stream with cycle prevention and customizable directive syntax
- **Input** — 'D1' is the entry file; 'D2' lists files to omit; 'D3' and 'D4' define directive and path delimiters; 'B' controls fallback behavior in `nx_file_store`; 'N' controls diagnostics
- **Mechanism**
  - Parse delimiter specification/Split 'D4' using 'D3'; extract file-exclusion separator, directive sigil, directory separator, extension separator, and directive name; warn if more than five fields are provided
  - Validate delimiters/Each delimiter is checked via `nx_delim_sep` to ensure it is safe and unambiguous
  - Load root file/Resolve and register the root file using `nx_file_store`; abort if unreadable
  - Apply exclusion list/Split 'D2' using the exclusion separator; resolve each excluded file and update the drop-prefix to prevent re-inclusion
  - Construct directive regex/Build patterns matching the directive sigil, directive name, and filename argument
  - Recursive expansion/'stk[]' accumulates text fragments; 'trk[]' tracks pending include files; each include spawns a new DFS branch
  - Directive handling/If a line matches the directive:
    - **Prefix** — Extract text before the directive
    - **Argument** — Extract filename argument and attempt resolution via `nx_file_store`
    - **Branch** — If new file, push prefix and queue file for DFS expansion; otherwise emit prefix only
  - Non-directive lines/Preserved verbatim unless empty or whitespace-only
  - DFS flattening/After all recursive expansions, `nx_dfs(stk)` linearizes the include tree into a stable output order
  - Emission/Final merged text is printed in DFS order, preserving all non-empty fragments

Output ⇝ Prints a fully expanded text stream with all include directives resolved; returns '-1' on delimiter or file-resolution failure

Use Case ⇝ Core of the Nx preprocessor: enables modular documents, chained configuration files, recursive overlays, and controlled include semantics with cycle prevention and customizable syntax

VII Examples

Basic include expansion

State ⇝ D1='main.txt', directive sigil '#', directive name 'nx_include'

Operation ⇝ Expand all '#nx_include file' directives recursively

Expected Result ⇝ Merged output printed with all includes resolved in DFS order

```
nx_file_merge("main.txt", "", ",", "#,/,.,nx_include", 1, 1)
```

Exclude specific files during expansion

State ⇝ D2='secret.cfg,debug.cfg'

Operation ⇝ If encountered after a directive, these files are ignored

Expected Result ⇝ Expansion proceeds without merging excluded files

```
nx_file_merge("main.txt", "secret.cfg,debug.cfg", ",",
↪"#,/,.,nx_include", 1, 1)
```

VIII Shell Modules

VIII Schema Compiler

$$
\begin{aligned}
S_{\text{base}} &= 12 && \text{(fixed slots for keywords, flags, arrays)} \\
S_{\text{group}} &= 3 && \text{(leader type, leader symbol, end index)} \\
G &= \text{number of groups declared in the schema} \\
S_{\text{groups}} &= G \cdot S_{\text{group}} && \text{(each group contributes exactly 3 slots)} \\
S_{\text{total}} &= S_{\text{base}} + S_{\text{groups}} && \text{(final stride for all category walks)}
\end{aligned}
$$

Schema Compiler Arguments

- D1 ⇝ Option schema string to tokenize (raw input)
- V ⇝ Array to populate with parsed flags, keywords, arrays, groups
- D2 ⇝ Separator of separators (defaults to comma)
- N ⇝ Runtime toggles string ‘dbg,ovr’
- D3 ⇝ List of separator characters (key, array, group, long/short mode)

```
function nx_shell_opts(D1, V, D2, N, D3)
```

Runtime Toggles Positions in N used by `nx_shell_opts`

Position	Variable	Meaning in `nx_shell_opts`
‘1’	dbg	Verbosity / debug level (controls warnings and logs)
‘2’	ovr	Override mode for group type resolution (see below)

ovr semantics (group type override)

- **Default** → When `ovr = 0`, existing group type is preserved when extending a group
- **Override** → When `ovr = 1`, the group type symbol is replaced with the new one
- **Code path** → On group extension, `ovr` controls whether `V[tmpb + strde]` is updated or left intact

debug levels

- **0** → silent
- **>0** → errors only
- **>1** → warnings
- **>2** → verbose audit
- **>3** → mapping summary
- **>4** → full dump

Variable Positions in D3

Index	Default	Variable	Meaning
–	‘;’	ds	Default delimiter string
1	‘:’	ks	Key separator
2	‘@’	fas	Appendable flag array separator
3	‘#’	kas	Appendable keyword array separator
4	‘<’	go	Begin group marker
5	‘>’	gc	End group marker
6	‘ ’	lo	Begin or continue long option mode
7	‘;’	lc	End long option mode

Short vs Long Option Group Leaders

- **Short form** ⟿ Input: ‘alpha<beta␣gamma>’ → Flags = a, l, p, h; Group leader = a; Members = b, e, t, a, gamma
- **Long form** ⟿ Input: ‘␣alpha<beta␣gamma>’ → Group leader = alpha; Members = beta, gamma
- **Key distinction** ⟿ Leading space (‘lo’) preserves token as long option; without space, token explodes into flags and first flag becomes leader

Anchors spaced by S_{total}

Category	Base index	Walk by
Keywords	1	$+ S_{\text{total}}$
Flags	4	$+ S_{\text{total}}$
Flag arrays	7	$+ S_{\text{total}}$
Keyword arrays	10	$+ S_{\text{total}}$
Groups (triplets)	13	$+3$ per group header, values spaced by S_{total}

Schema Compiler — Internal Semantics

- **Purpose** ⟿ Compile schema string into a stride-aligned registry
- **Categories** ⟿ Registers keywords, flags, flag arrays, keyword arrays, and groups
- **Stride** ⟿ Computes total stride $S_{\text{total}} = 12 + S_{\text{groups}}$
- **Groups** ⟿ Leader + members stored in triplets beginning at index 13
- **Long Option Mode** ⟿ Preserves multi-character tokens when `lo` is active
- **Validation** ⟿ Ensures separators are single, non-alphanumeric characters
- **Output** ⟿ Populates registry V with canonical positions and namespace anchors

Category Anchors (Base Indices)

- **Keywords** → Base index 1, stride $+S$
- **Flags** → Base index 4, stride $+S$
- **Flag Arrays** → Base index 7, stride $+S$
- **Keyword Arrays** → Base index 10, stride $+S$
- **Groups** → Base index 13, triplets spaced by 3, values spaced by S

VIII Examples

Register simple flags

- **State** → D1 = ‘abc’, V = {}, N = ‘3,0’
- **Operation** → Compile schema into flag entries
- **Expected Result** → a, b, c registered as flags at base index 4, spaced by S_{total}

```
nx_shell_opts("abc", V, ",", "3,0", "")
```

Register keyword arguments (short form)

- **State** → D1 = ‘k:x:y’, V = {}, N = ‘3,0’
- **Operation** → Use `ks = :` to mark keyword positions
- **Expected Result** → k, x, y registered as keyword entries at base index 1, spaced by S_{total}

```
nx_shell_opts("k:x:y", V, ",", "3,0", "")
```

Register long-form keyword

- State ⇝ D1=‘ alpha:beta’, V={}, N=‘3,0’
- Operation ⇝ Leading space (lo) preserves alpha as a single token
- Expected Result ⇝ alpha registered as a keyword entrie at base index 1 beta as a flag at base index 4

```
nx_shell_opts(" alpha:beta", V, ",", "3,0", "")
```

Register flag arrays

- State ⇝ D1=‘a@b@c’, V={}, N=‘3,0’
- Operation ⇝ Use fas = @ to mark appendable flag-array entries
- Expected Result ⇝ a, b, c registered at base index 7, spaced by S_{total}

```
nx_shell_opts("a@b@c", V, ",", "3,0", "")
```

Register short-form option group

- State ⇝ D1=‘a<b c>’, V={}, N=‘3,0’
- Operation ⇝ Group leader a with members b, c
- Expected Result ⇝ Group header triplet starts at index 13; members stored at group base + S_{total} multiples

```
nx_shell_opts("a<b c>", V, ",", "3,0", "")
```

Register long-form option group

- **State** ⟿ `D1=‘ alpha<beta gamma>’, V={}, N=‘3,0’`
- **Operation** ⟿ Leading space preserves `alpha` as group leader; `beta`, `gamma` as members
- **Expected Result** ⟿ Group leader `alpha` stored in group header; members spaced by S_{total}

```
nx_shell_opts(" alpha<beta gamma>", V, ",", "3,0", "")
```